

MODULE 1 · FOUNDATIONS

kubectl & Imperative vs Declarative

One CLI, two ways to manage every object

Every lab from here on talks to the cluster through kubectl

kubectl is the only client you'll use in this course. It's a thin wrapper over the Kubernetes API — every command it sends, the API server would accept from any other client too.

Before you build anything, get fluent in how commands are shaped, how to look up what you don't remember, and the two ways to change cluster state: **imperative** and **declarative**.

BY THE END YOU CAN

- Parse any kubectl command by grammar
- Look up fields with explain, not the browser
- Generate YAML instead of hand-writing it
- Filter objects with label selectors

CONCEPT

One grammar: **verb** → **resource** → **name** → **flags**

EVERY KUBECTL COMMAND FOLLOWS THE SAME SHAPE



`kubectl get pod nginx -n training -o wide` — verb `get`, resource `pod`, name `nginx`, flags `-n` and `-o`.

Find the resource, then read its shape

TASK	COMMAND	WHAT YOU GET
List every resource type	<code>kubectl api-resources</code>	Name, short name (<code>po</code> , <code>deploy</code>), namespaced?
Read a field's shape	<code>kubectl explain pod.spec.containers</code>	Built-in docs for that exact API version

MENTAL MODEL

`explain` is the API's own field-level manual — it's always in sync with your cluster's version, unlike a bookmarked web page.

--dry-run builds the full YAML for you

```
# kubectl create deployment web2 --image=nginx:1.25 --replicas=2 -n training --dry-run=client -o yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  creationTimestamp: null
  labels:
    app: web2
  name: web2
  namespace: training
spec:
  replicas: 2
  selector:
    matchLabels:
      app: web2
  strategy: {}
  template:
    metadata:
      creationTimestamp: null
      labels:
        app: web2
    spec:
      containers:
        - image: nginx:1.25
          name: nginx
          resources: {}
status: {}
```

You typed only name, image, replicas, and -n — kubectl filled in labels, selector, and the strategy/resources/status stubs; --dry-run=client never touched the cluster to build it. Save it, edit it, then apply -f it — that's the bridge.

Imperative to explore, declarative to keep

IMPERATIVE — DO IT NOW	DECLARATIVE — DESCRIBE DESIRED STATE
<code>kubectl run / create / scale / label</code>	<code>kubectl apply -f file.yaml</code>
Fast — but re-running errors "already exists"	Idempotent — safe to re-run
The live cluster is the source of truth	The YAML file is the source of truth

THE BRIDGE TRICK

Turn any imperative command into a starting YAML file, without submitting it:

```
$ kubectl create deployment web2 --image=nginx:1.25 --replicas=2 \
  -n training --dry-run=client -o yaml > web2.yaml
```

SEE IT

apply is idempotent — diff shows why

CHANGE THE FILE, PREVIEW, THEN APPLY ONLY THE DIFF

edit web2.yaml



kubectl diff -f
preview only



kubectl apply -f



cluster == file

Re-running `apply -f` with no changes does nothing. Running `create` twice on the same name errors instead.

How Services and controllers find their Pods

A label is just a key/value tag on an object. A **selector** is a filter over those tags — `-l tier=frontend` narrows any get to matching objects only.

This isn't just a CLI convenience: Services and Deployments use the exact same selector mechanism internally to find the Pods they own (Lab 2.3+).



Where people trip

- **create isn't idempotent.** Run it twice on the same name and it errors — apply won't.
- **Forgetting -n.** No namespace flag silently targets default.
- **get hides the reason.** A status like Pending needs describe to explain why.
- **Hand-editing live objects.** `kubectl edit` drifts the cluster away from your YAML source of truth.

Commands to keep at hand

```
$ kubectl api-resources           # discover resource types
$ kubectl explain pod.spec.containers # field-level docs
$ kubectl get pods,deploy -n training # summary of multiple kinds
$ kubectl describe deployment NAME -n training # events + full detail
$ kubectl create ... --dry-run=client -o yaml # generate, then edit
$ kubectl diff -f FILE.yaml          # preview before apply
$ kubectl get pods -l KEY=VALUE      # filter by label
```

Tip: describe before you trust get — the summary column never tells you why.

RECAP

kubectl in one breath

- Grammar: verb · resource · name · flags — same shape every time
- `api-resources` discovers types; `explain` documents fields
- Imperative = do it now; declarative = `apply -f` a file you own
- `--dry-run=client -o yaml` bridges the two
- Label selectors (`-l`) filter — and power Services/controllers

HANDS-ON

Now run Lab 1.1
`labs/1.1/`

Next: [1.2 — Inspect Your Cluster](#)